

DISCIPLINA DE DESENVOLVIMENTO WEB | SENAC SAO PAULO

AZURE

MANUAL DE DEPLOY NO MICROSOFT AZURE

SenacGames

Publicacao completa de uma aplicacao ASP.NET Core
com API, MVC e Azure SQL Database

.NET 8

ASP.NET Core

EF Core

Azure App Service

Azure SQL

Nivel: Iniciante | .NET 8.0 | Visual Studio 2022 | Microsoft Azure

INFORMAÇÕES DO DOCUMENTO

Título:	Manual de Deploy no Microsoft Azure
Subtítulo:	Publicação completa da aplicação SenacGames utilizando Azure App Service, Azure SQL Database e Entity Framework Core
Projeto:	SenacGames
Tecnologias:	.NET 8 · ASP.NET Core MVC · ASP.NET Core Web API Entity Framework Core · Azure App Service · Azure SQL Database Visual Studio 2022 · Microsoft Azure
Nível:	Iniciante
Versão .NET:	.NET 8.0
Finalidade:	Material didático educacional — Senac São Paulo
Disciplina:	Desenvolvimento Web com ASP.NET Core
Observação:	Material criado para uso educacional. Os termos, créditos e funcionalidades do Microsoft Azure podem ser atualizados periodicamente. Sempre verifique as condições atuais no portal oficial da Microsoft.

Este documento é material de apoio educacional. Não substitui a documentação oficial do Microsoft Azure. Sempre consulte docs.microsoft.com para informações atualizadas sobre os serviços Azure.

SUMÁRIO

- 1 Introdução ao Deploy
- 2 Pré-Requisitos
- 3 Como Obter o Benefício Azure for Students
- 4 Controle de Créditos e Custos no Azure
- 5 Planejamento dos Recursos
- 6 Criar o Grupo de Recursos
- 7 Criar o SQL Server no Azure
- 8 Configurar o Firewall do Azure SQL
- 9 Obter a Connection String do Azure SQL
- 10 Preparar o Projeto para Produção
- 11 Configurar a Connection String para o Entity Framework
- 12 Executar as Migrations no Azure SQL
- 13 Validar o Banco de Dados no Azure
- 14 Criar o App Service para a SenacGames.API
- 15 Configurar a SenacGames.API no Azure
- 16 Publicar a SenacGames.API pelo Visual Studio
- 17 Testar a API Hospedada
- 18 Configurar CORS na API
- 19 Configurar a Comunicação entre UI e API
- 20 Criar o App Service para a SenacGames.UI
- 21 Configurar a SenacGames.UI no Azure
- 22 Publicar a SenacGames.UI pelo Visual Studio
- 23 Teste Completo da Aplicação em Produção
- 24 Fluxo Completo de Comunicação
- 25 Problemas Comuns e Soluções
- 26 Monitoramento e Logs
- 27 Segurança e Boas Práticas

28 Encerramento do Projeto e Economia de Créditos

29 Checklist Final

— Referência Rápida de Comandos

— Informações de Referência do Projeto

- Documento: Guia Definitivo de Publicação em Nuvem Aplicação: SenacGames — ASP.NET Core + Entity Framework Core + SQL Server Plataforma: Microsoft Azure Nível: Iniciante Versão do .NET: 8.0

1 Introdução ao Deploy

O que significa fazer deploy?

Quando você desenvolve uma aplicação no seu computador, ela existe **apenas localmente**. Somente você consegue acessá-la, e somente enquanto o computador está ligado e o Visual Studio está rodando.

Deploy é o processo de publicar sua aplicação em um servidor na internet, de forma que qualquer pessoa com acesso à URL possa utilizá-la, a qualquer hora, de qualquer lugar do mundo.

Diferença entre executar localmente e publicar na nuvem

Aspecto	Localmente	Na Nuvem (Azure)
Acesso	Só você, no seu computador	Qualquer pessoa com a URL
Disponibilidade	Só quando o VS está rodando	24h por dia, 7 dias por semana
URL	localhost:5223	https://seuapp.azurewebsites.net
Banco de dados	SQL Server LocalDB (só local)	Azure SQL Database (nuvem)
Custo	Gratuito	Consome créditos Azure

O que é o Microsoft Azure?

O **Microsoft Azure** é a plataforma de computação em nuvem da Microsoft. Ele oferece centenas de serviços para hospedar aplicações, bancos de dados, arquivos, e muito mais. É utilizado por empresas do mundo inteiro para hospedar sistemas em produção.

Neste tutorial, utilizaremos o Azure para hospedar tanto a API quanto a interface web do SenacGames, além do banco de dados SQL Server.

O que será criado durante este tutorial?

Ao final deste tutorial, você terá criado a seguinte infraestrutura no Azure:

- | | | |
|---|--------------------------------------|--|
| 1 | 1 Grupo de Recursos (Resource Group) | -> Agrupa todos os recursos do projeto |
| 2 | 1 SQL Server Logico | -> Servidor de banco de dados no Azure |
| 3 | 1 Azure SQL Database | -> Banco de dados SenacGamesDb |
| 4 | 1 Azure App Service para a API | -> Hospeda a SenacGames.API |
| 5 | 1 Azure App Service para a UI | -> Hospeda a SenacGames.UI |

Arquitetura Final no Azure

Veja abaixo como a aplicação ficara estruturada apos o deploy:

```

1      INTERNET
2      |
3      v
4      +-----+
5      | SenacGames.UI |
6      | ASP.NET Core MVC |
7      | Azure App Service |
8      | (Portal do usuario) |
9      +-----+
10     |
11     HTTPS / HttpClient
12     (URL publica da API)
13     |
14     v
15     +-----+
16     | SenacGames.API |
17     | ASP.NET Core API |
18     | Azure App Service |
19     | (Backend + Swagger) |
20     +-----+
21     |
22     Entity Framework Core
23     (Connection String)
24     |
25     v
26     +-----+
27     | Azure SQL Database |
28     | SQL Server |
29     | (Banco de dados) |
30     +-----+

```

Pontos importantes sobre essa arquitetura:

- A **UI** **nao** **acessa** o **banco** **diretamente** — ela se comunica apenas com a API.
- A **API** **e** a **unica** **responsavel** por acessar o banco de dados via Entity Framework Core.
- A **UI** **e** a **API** **sao** **hospedadas** **separadamente** — cada uma em seu proprio App Service, com sua propria URL.
- Todo o trafego entre UI e API utiliza **HTTPS** (comunicacao segura e criptografada).
- O **banco de dados** **fica** **protegido** — ele so e acessivel pela API, nao diretamente pela internet.

2 Pré-Requisitos

Antes de iniciar o processo de deploy, certifique-se de que todos os itens abaixo estão prontos.

O que você precisa ter

- Conta Microsoft — pode ser pessoal (@outlook.com, @hotmail.com) ou institucional.
- E-mail institucional do Senac — o e-mail @senac.edu.br será necessário para ativar o Azure for Students.
- Visual Studio 2022 com o workload **"Desenvolvimento Web e ASP.NET"** instalado.
- .NET 8 SDK instalado no computador.
- Projeto SenacGames funcionando corretamente no ambiente local.
- SQL Server LocalDB funcionando e banco de dados local criado.
- Migrations criadas e aplicadas no banco local.
- Conexão com a internet estável.
- Navegador atualizado (Google Chrome ou Microsoft Edge recomendados).

Checklist de funcionamento local

Antes de começar o deploy, execute a aplicação localmente e confirme cada item:

```
1 [ ] Projeto compila sem erros
2 [ ] API inicia corretamente (Visual Studio mostra URL no console)
3 [ ] Swagger abre corretamente em http://localhost:5223/swagger
4 [ ] UI inicia corretamente e abre no navegador
5 [ ] Banco de dados local existe e possui tabelas
6 [ ] Login funciona (admin@senacgames.com / Admin@123)
7 [ ] Listagem de Games funciona na UI
8 [ ] CRUD de Games funciona (criar, editar, excluir)
9 [ ] Categorias aparecem corretamente
```

- **IMPORTANTE:** Não inicie o processo de deploy enquanto a aplicação não estiver funcionando 100% localmente. Se há erros locais, esses mesmos erros aparecerão no Azure — e serão muito mais difíceis de diagnosticar. Resolva todos os problemas locais antes de continuar.

3

Como Obter o Benefício Azure for Students

O que é o Azure for Students?

O **Azure for Students** é um benefício educacional gratuito oferecido pela Microsoft para estudantes universitários. Ele oferece acesso gratuito ao Microsoft Azure por 12 meses, com um crédito para uso nos serviços da plataforma.

- **DICA:** Consulte a página oficial <https://azure.microsoft.com/pt-br/free/students/> para verificar os termos e créditos atuais disponíveis para estudantes. A Microsoft pode atualizar os valores e condições periodicamente.

Passo a passo para ativar o Azure for Students

■ Passo 1 — Acesse a página oficial

1. Abra seu navegador e acesse: <https://azure.microsoft.com/pt-br/free/students/>
2. Clique no botão **"Ativar agora"** ou **"Começar gratuitamente"**.

■ Passo 2 — Entre com sua conta Microsoft

1. Você será redirecionado para a página de login da Microsoft.
2. Se já possui uma conta Microsoft, insira seu e-mail e senha.
3. Se não possui, clique em **"Criar uma"** e siga as instruções.

- **DICA:** Se o seu e-mail institucional senac@senac.edu.br ainda não possui uma conta Microsoft associada, você pode criar uma durante esse processo. Use preferencialmente o e-mail institucional para facilitar a validação estudantil.

■ Passo 3 — Verificação de elegibilidade estudantil

1. O Azure solicitará verificação de que você é estudante.
2. Informe seu **e-mail institucional**: `seu.nome@senac.edu.br`
3. A Microsoft enviará um **código de verificação** para esse e-mail.
4. Abra o e-mail institucional, copie o código recebido e cole no campo indicado.
5. Confirme a verificação.

■ Passo 4 — Preencha o formulário de ativação

1. Preencha as informacoes solicitadas:
 - Nome completo.
 - Pais/Regiao: **Brasil**.
 - Data de nascimento.
 - Numero de telefone para verificacao adicional (se solicitado).
2. Leia e aceite os termos de uso.
3. Clique em "**Verificar identidade academica**" ou equivalente.

■ Passo 5 — Aguarde a confirmacao

1. O processo pode levar alguns minutos.
2. Quando concluido, voce vera uma mensagem confirmando que o Azure for Students foi ativado.
3. Clique em "**Ir para o Portal do Azure**" ou acesse diretamente: <https://portal.azure.com>

Como verificar se o beneficio foi ativado corretamente

■ Verificando a assinatura

1. Acesse <https://portal.azure.com>
2. No menu a esquerda ou na barra de pesquisa, busque por "**Assinaturas**" (*Subscriptions*).
3. Voce devera ver uma assinatura chamada "**Azure for Students**" listada.
4. Clique nela para ver os detalhes.

■ Verificando o credito disponivel

1. Na tela da assinatura **Azure for Students**, procure por "**Creditos**" ou "**Credits**".
2. Verifique o valor disponivel e a data de expiracao.

■ **IMPORTANTE:** Os termos e creditos do Azure for Students podem mudar. Sempre verifique as condicoes atuais diretamente no portal da Microsoft em <https://azure.microsoft.com/pt-br/free/students/>. Nao assuma valores especificos sem confirmar no portal.

Possiveis problemas e solucoes

■ E-mail institucional nao reconhecido

Problema: O Azure nao aceita seu e-mail senac@senac.edu.br.

Solucao:

- Certifique-se de que o e-mail esta digitado corretamente.
- Verifique se a caixa de entrada do e-mail institucional esta acessivel — voce precisara receber o codigo de verificacao.

- Em alguns casos, a Microsoft pode não reconhecer automaticamente a instituição. Nesse caso, entre em contato com a equipe do professor para orientações alternativas.

■ Conta Microsoft diferente do e-mail institucional

Problema: Você fez login com uma conta Microsoft pessoal (@outlook.com) mas precisa validar com o e-mail institucional.

Solução:

- Isso é normal. O processo de verificação aceita usar uma conta Microsoft pessoal para login, mas valida o status estudantil pelo e-mail institucional.
- Insira o e-mail @senacedu.sp.br somente no campo de **verificação acadêmica**.

■ Assinatura não aparece no Portal

Problema: Você completou o processo mas a assinatura Azure for Students não aparece.

Solução:

- Aguarde alguns minutos e atualize a página.
- Certifique-se de estar logado com a conta Microsoft correta.
- Acesse <https://portal.azure.com> > **Assinaturas** e verifique se há alguma assinatura listada.

■ Benefício indisponível

Problema: A Microsoft indica que você não é elegível.

Possíveis causas:

- O benefício já foi utilizado anteriormente com este e-mail.
- O e-mail institucional não é reconhecido pelo programa.
- Restrições geográficas ou de conta.

Solução: Entre em contato com seu professor para orientação sobre alternativas.

4

Controle de Créditos e Custos no Azure

- **IMPORTANTE:** Esta seção é fundamental. Ignorar o controle de custos pode resultar em consumo inesperado dos seus créditos estudantis.

O que pode consumir seus créditos?

Cada recurso criado no Azure pode gerar custos, mesmo que você não o esteja utilizando ativamente no momento. Os principais geradores de custo neste tutorial são:

Recurso	Geracao de Custo
Azure SQL Database	Gera custo continuamente enquanto existe
Azure App Service (API)	Gera custo conforme o plano escolhido
Azure App Service (UI)	Gera custo conforme o plano escolhido
SQL Server (logico)	Pode gerar custo dependendo da configuracao

- **IMPORTANTE:** Criar um recurso e não utilizá-lo não significa que ele não gera consumo. Um banco de dados criado e deixado ativo continuará consumindo créditos mesmo sem receber conexões.

Como acompanhar o consumo de créditos

■ Acessando Cost Management

1. Acesse <https://portal.azure.com>
2. Na barra de pesquisa do portal, digite "**Cost Management**".
3. Clique em "**Gerenciamento de Custos + Cobrança**".
4. No menu lateral, clique em "**Análise de custos**" (*Cost analysis*).
5. Você verá um painel com o consumo atual e histórico.

■ Verificando custos por recurso

1. Em **Cost Management**, clique em "**Análise de custos**".
2. No topo, altere o agrupamento para "**Recurso**" (*Resource*) para ver quanto cada serviço está consumindo.

3. Isso permite identificar quais recursos estão gerando mais gastos.

Como criar um alerta de orçamento

Criar alertas é uma das melhores formas de evitar surpresas.

1. Em **Cost Management**, clique em "**Orcamentos**" (*Budgets*).
2. Clique em "+ Adicionar".
3. Selecione o escopo como sua assinatura **Azure for Students**.
4. Defina um valor de orçamento mensal (exemplo: defina um valor abaixo do seu crédito total).
5. Em **Condições de alerta**, adicione:
 - **80% do orçamento** — para receber aviso antes de atingir o limite.
 - **100% do orçamento** — para receber aviso ao atingir o limite.
6. Informe seu e-mail para receber as notificações.
7. Clique em "**Criar**".

■ **RESULTADO ESPERADO:** Você receberá e-mails automáticos quando o consumo atingir os percentuais configurados, permitindo tomar ação antes de esgotar os créditos.

Boas práticas de economia

- **Escolha os planos mais econômicos** disponíveis para ambiente estudantil.
- **Exclua os recursos quando não precisar mais deles.** Ao final do semestre ou do projeto, remova todos os recursos criados.
- **Agrupe todos os recursos no mesmo Resource Group** — isso facilita a exclusão em massa depois.
- **Não crie recursos desnecessários.** Apenas crie o que está descrito neste tutorial.
- **Monitore o consumo semanalmente** durante o período em que a aplicação estiver hospedada.

5 Planejamento dos Recursos

Antes de criar qualquer recurso no Azure, organize o que sera criado. Isso evita confusao e garante que tudo ficara agrupado corretamente.

Tabela de recursos

Recurso	Nome sugerido	Funcao
Resource Group	rg-senacgames	Agrupar todos os recursos do projeto
SQL Server (logico)	sql-senacgames	Servidor logico do banco de dados
SQL Database	SenacGamesDb	Banco de dados da aplicacao
App Service — API	app-senacgames-api	Hospedar a SenacGames.API
App Service — UI	app-senacgames-ui	Hospedar a SenacGames.UI

Sobre nomes unicos no Azure

- **IMPORTANTE:** Alguns recursos no Azure exigem nomes globalmente unicos — ou seja, nenhum outro usuario no mundo pode ter um recurso com o mesmo nome. Isso se aplica principalmente aos App Services e ao SQL Server, pois seus nomes fazem parte de URLs publicas. Se o nome app-senacgames-api ja foi utilizado por outra pessoa, o Azure avisara que o nome nao esta disponivel.

■ Como resolver conflitos de nome

Use seu nome ou matricula como sufixo para garantir unicidade:

```
1 app-senacgames-api-luan
2 app-senacgames-ui-luan
3 sql-senacgames-luan
```

Ou utilize sua matricula:

```
1 app-senacgames-api-12345
2 app-senacgames-ui-12345
```

URLs que serao geradas

Apos criar os App Services, o Azure gerara URLs automaticas no formato:

```
1 | API: https://app-senacgames-api-luan.azurewebsites.net
2 | UI:  https://app-senacgames-ui-luan.azurewebsites.net
```

- DICA: Anote essas URLs assim que forem criadas. Voce precisara delas durante a configuracao da comunicacao entre UI e API.

6 Criar o Grupo de Recursos

O que é um Resource Group (Grupo de Recursos)?

Um **Resource Group** (Grupo de Recursos) é como uma pasta no Azure. Ele agrupa todos os recursos relacionados a um projeto em um único lugar, facilitando:

- **Gerenciamento:** todos os recursos aparecem juntos.
- **Exclusão em massa:** ao excluir o Resource Group, todos os recursos dentro dele são excluídos automaticamente.
- **Controle de custos:** você pode ver o custo total do projeto em um só lugar.

Todos os recursos do SenacGames devem ficar dentro do mesmo Resource Group.

Passo a passo para criar o Resource Group

■ Passo 1 — Acesse o Portal Azure

1. Abra <https://portal.azure.com> no navegador.
2. Faça login com sua conta Microsoft.

■ Passo 2 — Acesse "Grupos de Recursos"

1. Na barra de pesquisa no topo da página, digite: Grupos de recursos OU Resource groups.
2. Clique no resultado correspondente.

■ Passo 3 — Clique em "Criar"

1. Na tela de Grupos de Recursos, clique no botão "+ Criar" (ou "Create").

■ Passo 4 — Preencha os dados do Resource Group

Você verá um formulário com os seguintes campos:

Assinatura (Subscription):

- Selecione: **Azure for Students**

■ **IMPORTANTE:** Sempre selecione a assinatura Azure for Students. Se você selecionar outra assinatura (como "Pay-As-You-Go"), poderá ser cobrado em dinheiro real.

Grupo de recursos (Resource group):

- Nome: **rg-senacgames** (ou adicione seu nome: **rg-senacgames-luan**)

Regiao (Region):

- Selecione: **Brazil South** ou **East US**

■ DICA: Brazil South e o datacenter brasileiro (Sao Paulo). E uma boa escolha por ser geograficamente mais proximo. Porem, se encontrar problemas de disponibilidade de recursos nessa regio, East US e uma alternativa confiavel e economica.

■ Passo 5 — Revise e crie

1. Clique em **"Revisar + criar"** (*Review + create*).
2. Revise as informacoes apresentadas.
3. Clique em **"Criar"** (*Create*).
4. Aguarde alguns segundos. O Azure criara o Resource Group.

■ Passo 6 — Confirme a criacao

1. Voce vera a mensagem: **"A implantacao foi concluida"** (*Deployment succeeded*).
2. Clique em **"Ir para o recurso"** para ver o Resource Group criado.

■ RESULTADO ESPERADO: Um Grupo de Recursos (rg-senacgames) criado e visivel no Portal Azure, pronto para receber todos os recursos do projeto SenacGames.

7 Criar o SQL Server no Azure

SQL Server vs. SQL Database — qual a diferença?

No Azure SQL, existe uma separação importante:

Conceito	O que é
SQL Server (lógico)	É o <i>servidor</i> — funciona como um container que pode hospedar vários bancos
SQL Database	É o <i>banco de dados</i> propriamente dito — onde os dados ficam armazenados

Pense assim: o SQL Server é o prédio, e o SQL Database é um apartamento dentro desse prédio. Você precisará criar os dois.

- **DICA:** Ao criar o SQL Database pelo Portal Azure, você poderá criar o SQL Server lógico durante o mesmo processo. Não precisa criá-los separadamente.

Passo a passo para criar o SQL Database (e o SQL Server)

■ Passo 1 — Acesse "Bancos de Dados SQL"

1. Na barra de pesquisa do Portal Azure, digite: `SQL databases` ou `Bancos de dados SQL`.
2. Clique no resultado correspondente.
3. Clique em "+ Criar".

■ Passo 2 — Aba "Básico" — Configurações gerais

Você verá um formulário com várias abas. Preencha a aba "**Básico**":

Assinatura (Subscription):

- Selecione: **Azure for Students**

Grupo de recursos (Resource group):

- Selecione: **rg-senacgames** (o que você criou no passo anterior)

Nome do banco de dados (Database name):

- Digite: **SenacGamesDb**

Servidor (Server):

- Clique em **"Criar novo"** (*Create new*).

■ Passo 3 — Criar o SQL Server logico

Uma janela lateral sera aberta para criar o servidor. Preencha:

Nome do servidor (Server name):

- Digite: `sql-senacgames-luan` (substitua "luan" pelo seu nome)
- O Azure mostrara automaticamente a URL completa: `sql-senacgames-luan.database.windows.net`

Regiao (Location):

- Selecione a mesma regiao do Resource Group: **Brazil South** ou **East US**

Metodo de autenticao (Authentication method):

- Selecione: **"Usar autenticao SQL"** (*Use SQL authentication*)

■ **IMPORTANTE:** Neste tutorial utilizamos autenticao SQL (usuario e senha) por ser mais simples e direta. Nao selecione "Microsoft Entra-only authentication" para evitar complicacoes de configuracao.

Logon do administrador do servidor (Server admin login):

- Digite um nome de usuario. Exemplo: `adminsena`
- **Evite usar:** `admin`, `sa`, `root`, `administrator` — o Azure nao aceita esses nomes reservados.

Senha (Password):

- Crie uma senha forte. Ela deve conter:
- Letras maiusculas e minusculas
- Numeros
- Caracteres especiais
- Minimo de 8 caracteres
- Exemplo de formato (nao use exatamente este): `SenacGames@2025!`

■ **ATENCAO CRITICA:** Anote imediatamente o nome do servidor, o login e a senha em um local seguro (como um arquivo no seu computador ou gerenciador de senhas). Voce precisara dessas informacoes mais tarde para configurar a Connection String. Nunca salve essas credenciais em arquivos que serao enviados ao GitHub.

Clique em **"OK"** para confirmar a criacao do servidor.

■ Passo 4 — Selecionar ou configurar a Computacao + Armazenamento

De volta ao formulario do SQL Database:

Computacao + armazenamento (Compute + storage):

- Clique em "**Configurar banco de dados**" (*Configure database*).
- Para ambiente estudantil, procure a opção mais econômica disponível.
- O nível "**Basic**" ou "**Standard S0**" é geralmente suficiente para projetos de estudo.

■ **IMPORTANTE:** As opções e preços do Azure mudam frequentemente. Verifique no portal qual é a opção de menor custo disponível no momento. O objetivo é manter o consumo dentro dos seus créditos estudantis. Não selecione configurações de alta performance desnecessariamente.

Redundância do armazenamento de backup (Backup storage redundancy):

- Selecione: "**Armazenamento de backup com redundância local**" (*Locally redundant backup storage*) — é a opção mais econômica.

■ Passo 5 — Revise e crie

1. Clique em "**Revisar + criar**".
2. Revise todas as informações.
3. Clique em "**Criar**".
4. **Aguarde a implantação** — pode levar de 2 a 5 minutos.

■ **RESULTADO ESPERADO:** - Um SQL Server lógico criado (ex: sql-senacgames-luan.database.windows.net). - Um SQL Database chamado SenacGamesDb dentro desse servidor. - Ambos aparecem dentro do Resource Group rg-senacgames.

8 Configurar o Firewall do Azure SQL

Por que o banco SQL Azure bloqueia conexões externas?

Por padrão, o **Azure SQL Database bloqueia todas as conexões externas** — incluindo a sua. Isso é uma medida de segurança. Para que você consiga se conectar ao banco (tanto do seu computador quanto dos App Services), precisa configurar as regras de firewall.

Diferença entre os tipos de acesso

Tipo de acesso	Para que serve
IP do seu computador	Para executar migrations localmente apontando para o Azure SQL
Serviços do Azure	Para que o App Service da API consiga se conectar ao banco em produção

Passo a passo para configurar o Firewall

■ Passo 1 — Acesse o SQL Server criado

1. No Portal Azure, acesse "**Servidores SQL**" (*SQL servers*) ou busque pelo nome do servidor (ex: `sql-senacgames-luan`).
2. Clique no servidor para abrir seus detalhes.

■ Passo 2 — Acesse as configurações de Rede/Firewall

1. No menu lateral esquerdo do servidor SQL, procure por "**Rede**" (*Networking*) ou "**Firewalls e redes virtuais**" (*Firewalls and virtual networks*).
2. Clique nessa opção.

■ Passo 3 — Permitir acesso do seu computador

1. Na seção "**Regras de firewall**" (*Firewall rules*), clique em "**+ Adicionar IP do cliente**" (*Add client IP*).
2. O Azure detectará automaticamente o IP público do seu computador e adicionará uma regra.
3. Você verá uma linha adicionada com seu IP (ex: `191.xxx.xxx.xxx`).

- **DICA:** Cada vez que você mudar de rede (ex: sair do Senac e ir para casa), seu IP pode mudar. Se encontrar erros de conexão ao executar migrations, volte aqui e adicione o IP atual novamente.

■ Passo 4 — Permitir acesso dos serviços Azure

1. Na mesma tela de Rede/Firewall, procure a opção:

"Permitir que serviços e recursos do Azure acessem este servidor" (*Allow Azure services and resources to access this server*).

2. Ative essa opção (mude para **"Sim"** ou marque o checkbox, conforme a interface atual).

■ **IMPORTANTE:** Essa configuração é obrigatória para que o App Service da API consiga se conectar ao banco de dados em produção. Sem ela, a API hospedada no Azure não conseguirá acessar o SQL Database.

■ Passo 5 — Salve as configurações

1. Clique em **"Salvar"** (Save).
2. Aguarde a confirmação.

■ **RESULTADO ESPERADO:** - Seu IP está na lista de regras de firewall. - A opção "Permitir serviços Azure" está ativada. - Você conseguirá se conectar ao banco tanto do seu computador (para migrations) quanto pelo App Service (em produção).

9

Obter a Connection String do Azure SQL

O que é uma Connection String?

Uma **Connection String** (cadeia de conexão) é um texto que contém todas as informações necessárias para que sua aplicação se conecte ao banco de dados:

- Endereço do servidor
- Nome do banco de dados
- Usuário
- Senha
- Configurações de segurança

Onde encontrar a Connection String no Portal

■ Passo 1 — Acesse o SQL Database

1. No Portal Azure, busque por "**Bancos de dados SQL**" (*SQL databases*).
2. Clique no banco `SenacGamesDb`.

■ Passo 2 — Acesse as Connection Strings

1. No menu lateral esquerdo, clique em "**Cadeias de conexão**" (*Connection strings*).
2. Selecione a aba "**ADO.NET (autenticação do SQL)**" (*ADO.NET SQL authentication*).
3. Você verá a Connection String completa.

■ Formato da Connection String

A Connection String terá um formato semelhante a este:

```
1 Server=tcp:sql-senacgames-luan.database.windows.net,1433;  
2 Initial Catalog=SenacGamesDb;  
3 Persist Security Info=False;  
4 User ID={seu_usuario};  
5 Password={sua_senha};  
6 MultipleActiveResultSets=False;  
7 Encrypt=True;  
8 TrustServerCertificate=False;  
9 Connection Timeout=30;
```

■ Explicação de cada parte

Parte	Descricao
Server=tcp:...1433	Endereco e porta do servidor SQL no Azure
Initial Catalog=SenacGamesDb	Nome do banco de dados
User ID={seu_usuario}	Usuario administrador criado (ex: adminsenac)
Password={sua_senha}	Senha do usuario administrador
Encrypt=True	Exige comunicacao criptografada (TLS/SSL)
TrustServerCertificate=False	Valida o certificado do servidor
Connection Timeout=30	Tempo limite para estabelecer conexao (30 segundos)
MultipleActiveResultSets=False	Compativel com EF Core — manter como False para Azure SQL

■ Passo 3 — Copie e substitua as credenciais

1. Copie a Connection String exibida.
2. Substitua {seu_usuario} pelo login que voce criou (ex: adminsenac).
3. Substitua {sua_senha} pela senha que voce definiu.
4. **Guarde essa Connection String** em um local seguro no seu computador.

■ **ATENCAO CRITICA:** Nunca salve a Connection String com usuario e senha reais em arquivos que serao enviados ao GitHub. Isso exporia suas credenciais publicamente e poderia comprometer sua conta Azure. A Connection String com credenciais reais deve ficar apenas nas configuracoes do Azure App Service (que e privado e seguro), nunca no codigo-fonte.

10 Preparar o Projeto para Producao

Entendendo os ambientes

Sua aplicacao precisa funcionar em dois ambientes diferentes:

Ambiente	Onde roda	Banco de dados	URL da API
Desenvolvimento	Seu computador	LocalDB local	http://localhost:5223
Producao	Azure	Azure SQL Database	https://app-senacgame s-api.azurewebsites.n et

O ASP.NET Core possui um sistema de configuracao por ambiente que facilita isso.

Como o ASP.NET Core gerencia configuracoes por ambiente

O ASP.NET Core carrega configuracoes nessa ordem (a ultima ganha):

1. appsettings.json -> Configuracoes base (comuns a todos os ambientes)
2. appsettings.{ENVIRONMENT}.json -> Substitui configuracoes do ambiente especifico
3. Variaveis de ambiente do sistema -> Substitui tudo (usadas no Azure App Service)

Quando a aplicacao roda localmente com o Visual Studio, o ambiente e `Development`, entao `appsettings.Development.json` e carregado.

Quando a aplicacao roda no Azure (que configuraremos como `Production`), as **Application Settings** do App Service substituem os valores do `appsettings.json`.

Analise do projeto atual

■ SenacGames.API — appsettings.json atual

JSON

```
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "AllowedHosts": "*"
12 }
```

Esta Connection String aponta para o LocalDB local. **Em producao, precisamos substitui-la** pela Connection String do Azure SQL — e faremos isso nas configuracoes do App Service, sem alterar o codigo.

■ SenacGames.UI — Como a URL da API e resolvida

A UI utiliza a classe `ApiEndpointResolver` para descobrir a URL da API. Esse resolver funciona da seguinte forma:

```
1  1. Tenta ler o launchSettings.json da API (funciona apenas localmente)
2    |
3    v (Se nao encontrar ou nao existir...)
4  2. Tenta ler a secao ApiSettings.BaseUrl do appsettings.json da UI
5    |
6    v (Se nao encontrar...)
7  3. Retorna null (a aplicacao nao conseguiu chamar a API)
```

Em producao no Azure:

- O arquivo `launchSettings.json` **nao existe** (nao e publicado).
- O resolver tentara ler `ApiSettings.BaseUrl` do `appsettings.json`.

Isso significa que devemos adicionar a configuracao `ApiSettings.BaseUrl` ao `appsettings.json` da UI ou configura-la nas Application Settings do App Service.

Alteracoes recomendadas no projeto

■ Alteracao 1 — Adicionar suporte a `ApiSettings.BaseUrl` no `appsettings.json` da UI

Arquivo: `SenacGames.UI/appsettings.json`

Motivo: Em producao, a UI nao tem acesso ao `launchSettings.json` da API (esse arquivo nao e publicado). O `ApiEndpointResolver` ja possui logica para ler `ApiSettings.BaseUrl` do `appsettings.json` — mas a chave precisa estar presente. Adicionando essa configuracao no `appsettings.json` com a URL local e sobrescrevendo no Azure com a URL de producao, o sistema fica bem organizado e funciona nos dois

ambientes.

Como alterar:

Abra `SenacGames.UI/appsettings.json` e adicione a seção `ApiSettings`:

Antes:

JSON

```
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4    },
5    "Logging": {
6      "LogLevel": {
7        "Default": "Information",
8        "Microsoft.AspNetCore": "Warning"
9      }
10   },
11   "AllowedHosts": "*"
12 }
```

Depois:

JSON

```
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4    },
5    "ApiSettings": {
6      "BaseUrl": "http://localhost:5223/"
7    },
8    "Logging": {
9      "LogLevel": {
10       "Default": "Information",
11       "Microsoft.AspNetCore": "Warning"
12     }
13   },
14   "AllowedHosts": "*"
15 }
```

Como isso melhora a separacao entre ambientes:

- Localmente: a UI usa `http://localhost:5223/` (do `appsettings.json`).
- No Azure: a URL é sobrescrita pelas Application Settings do App Service com a URL real de produção.

■ Alteracao 2 — Habilitar Swagger em producao (recomendado para validacao)

Arquivo: `SenacGames.API/Program.cs`

Motivo: Atualmente o Swagger esta configurado para rodar apenas em `Development`. No Azure (configurado como `Production`), o Swagger nao estara acessivel. Sem o Swagger, fica dificil testar se a API esta funcionando corretamente apos o deploy.

Como alterar:

Localize este bloco no `Program.cs` da API:

CSHARP

```
1 | if (app.Environment.IsDevelopment())
2 | {
3 |     // Swagger so e habilitado em ambiente de desenvolvimento
4 |     app.UseSwagger();
5 |     app.UseSwaggerUI();
6 | }
```

Substitua por:

CSHARP

```
1 | // Habilita Swagger em todos os ambientes (Development e Production)
2 | // NOTA: Em projetos corporativos reais, o Swagger e normalmente desabilitado em produca
3 | // Para este projeto educacional, habilitamos em producao para facilitar os testes.
4 | app.UseSwagger();
5 | app.UseSwaggerUI();
```

Salve o arquivo.

11 Configurar a Connection String para o Entity Framework

Contexto do projeto

Antes de executar o `Update-Database`, precisamos entender como o Entity Framework esta configurado no `SenacGames`:

Componente	Localizacao
DbContext	<code>SenacGames.Infrastructure/Context/SenacGamesDbContext.cs</code>
Migrations	<code>SenacGames.Infrastructure/Migrations/</code>
Startup Project	<code>SenacGames.API</code> (le a Connection String)
Connection String	<code>SenacGames.API/appsettings.json</code>

O Entity Framework precisa de dois projetos para funcionar:

- **Projeto padrao** (`-Project`): onde o DbContext e as Migrations estao — `SenacGames.Infrastructure`.
- **Startup Project** (`-StartupProject`): onde a `Program.cs` configura o DbContext com a Connection String — `SenacGames.API`.

Preparando a Connection String para o Azure

Para que o `Update-Database` crie as tabelas no **Azure SQL** (e nao no LocalDB local), precisamos que a `SenacGames.API` use temporariamente a Connection String do Azure.

■ **IMPORTANTE:** Existem duas formas de fazer isso. Escolha UMA e use apenas durante o processo de migration. Depois reverta para o valor original.

■ Opcao A — Alterar temporariamente o appsettings.json (mais simples)

1. Abra o arquivo `SenacGames.API/appsettings.json`.
2. Substitua a Connection String do LocalDB pela Connection String do Azure SQL.

Antes:

JSON

```
1 | {
2 |   "ConnectionStrings": {
3 |     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4 |   }
5 | }
```

Depois (temporariamente):

JSON

```
1 | {
2 |   "ConnectionStrings": {
3 |     "DefaultConnection": "Server=tcp:sql-senacgames-luan.database.windows.net,1433;Initi
4 |   }
5 | }
```

- **ATENCAO CRITICA:** Apos executar o Update-Database com sucesso, REVERTA imediatamente o appsettings.json para a Connection String do LocalDB. Nunca faca commit dessa alteracao para o GitHub com credenciais reais.

■ Opcao B — Usar variavel de ambiente temporaria (mais segura)

No Console do Gerenciador de Pacotes do Visual Studio, antes de executar o `Update-Database`, configure uma variavel de ambiente:

POWERSHELL

```
1 | $env:ConnectionStrings__DefaultConnection = "Server=tcp:sql-senacgames-luan.database.win
```

Essa variavel existe apenas na sessao atual do PowerShell e nao fica salva em nenhum arquivo.

- **DICA:** A Opcao B e mais segura porque nao corre o risco de fazer commit acidental com credenciais reais. No entanto, exige que o comando seja executado no mesmo terminal onde a variavel foi definida.

12 Executar as Migrations no Azure SQL

- **IMPORTANTE:** Não crie novas migrations apenas para publicar no Azure. As migrations existentes em `SenacGames.Infrastructure/Migrations/` já contêm toda a estrutura do banco. O comando `Update-Database` as aplicará ao Azure SQL.

Fluxo correto

```
1 | Migrations existentes (SenacGames.Infrastructure)
2 |     +
3 | Connection String -> Azure SQL Database
4 |     |
5 |     v
6 |     Update-Database
7 |     |
8 |     v
9 | Estrutura criada no Azure SQL
10 | (tabelas, indices, constraints)
11 |     |
12 |     v
13 | SeedData.SeedAsync()
14 | (executado automaticamente na
15 | primeira inicializacao da API)
```

Método 1 — Via Console do Gerenciador de Pacotes (Visual Studio)

■ Passo 1 — Abra o Console do Gerenciador de Pacotes

1. No Visual Studio, clique no menu **"Ferramentas"** (*Tools*).
2. Clique em **"Gerenciador de Pacotes NuGet"** (*NuGet Package Manager*).
3. Clique em **"Console do Gerenciador de Pacotes"** (*Package Manager Console*).
4. O console será aberto na parte inferior do Visual Studio.

■ Passo 2 — Configure os projetos corretamente

No Console do Gerenciador de Pacotes, há dois seletores importantes no topo:

- **"Projeto Padrão"** (*Default project*): selecione `SenacGames.Infrastructure`

- **IMPORTANTE:** O Startup Project do Visual Studio (configurado pelo botão de play verde) deve estar definido como SenacGames.API. Verifique isso na barra superior do Visual Studio — o projeto exibido ao lado do botão de play deve ser SenacGames.API.

■ Passo 3 — Confirme a Connection String

Antes de executar o comando, confirme que a `SenacGames.API/appsettings.json` já contém a Connection String do Azure SQL (conforme descrito na seção anterior).

■ Passo 4 — Execute o comando

No Console do Gerenciador de Pacotes, digite:

POWERSHELL

```
1 | Update-Database -Project SenacGames.Infrastructure -StartupProject SenacGames.API
```

Pressione **Enter**.

Explicação de cada parâmetro:

Parâmetro	Valor	Descrição
-Project	SenacGames.Infrastructure	Onde estão as Migrations e o DbContext
-StartupProject	SenacGames.API	Onde está o Program.cs que configura a Connection String

■ Passo 5 — Acompanhe a execução

O console exibirá mensagens de progresso semelhantes a:

```
1 | Build started...
2 | Build succeeded.
3 | info: Microsoft.EntityFrameworkCore.Database.Command[20101]
4 |       Executed DbCommand (XXms) ...
5 |       CREATE TABLE [dbo].[__EFMigrationsHistory] ...
6 | Applying migration '20260521191810_Inicial'.
7 | Applying migration '20260521193849_NomeDaMigration'.
8 | Done.
```


- **RESULTADO ESPERADO:** A mensagem "Done." indica que as migrations foram aplicadas com sucesso e o banco foi criado no Azure SQL.

- **ERRO COMUM:** Se aparecer algo como "Login failed for user" ou "Cannot open server", verifique: - A Connection String esta correta (usuario, senha e nome do servidor)? - O firewall do Azure SQL permite o seu IP atual? - A Connection String esta no appsettings.json do projeto SenacGames.API?

Metodo 2 — Via Terminal (dotnet CLI)

Se preferir usar o terminal, abra o **PowerShell** ou o **Terminal do Visual Studio** e execute:

POWERSHELL

```
1 | dotnet ef database update --project SenacGames.Infrastructure --startup-project SenacGam
```

- **DICA:** Para usar o comando dotnet ef, e necessario ter o EF Core Tools instalado globalmente. Se encontrar o erro 'dotnet-ef' was not found, execute: ``powershell dotnet tool install --global dotnet-ef``

Apos executar as migrations — Reverta a Connection String

Depois de confirmar que as migrations foram aplicadas com sucesso:

1. Reverta o `SenacGames.API/appsettings.json` para a Connection String do LocalDB:

JSON

```
1 | {
2 |   "ConnectionStrings": {
3 |     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4 |   }
5 | }
```

2. A Connection String de producao ficara **apenas** nas Application Settings do Azure App Service (configurada nas proximas secoes).

13 Validar o Banco de Dados no Azure

Após executar o Update-Database, verifique se o banco foi criado corretamente.

Opção 1 — Verificação pelo Portal Azure (Query Editor)

O Portal Azure oferece um editor de queries diretamente no navegador, sem necessidade de instalar ferramentas adicionais.

■ Passo 1 — Acesse o banco de dados

1. No Portal Azure, acesse "**Bancos de dados SQL**".
2. Clique no banco SenacGamesDb.

■ Passo 2 — Abra o Query Editor

- No menu lateral esquerdo, clique em "**Editor de consultas**" (*Query editor (preview)*).
- Faça login com:
- **Login:** seu usuário administrador (ex: `adminsenac`)
- **Senha:** a senha que você criou

■ **DICA:** Se o Query Editor exibir um erro de firewall ao tentar fazer login, é porque seu IP não está autorizado. Volte a seção 8 (Configurar Firewall) e adicione seu IP atual.

■ Passo 3 — Verifique as tabelas criadas

Execute a seguinte query para ver todas as tabelas criadas:

SQL

```
1 SELECT TABLE_NAME
2 FROM INFORMATION_SCHEMA.TABLES
3 WHERE TABLE_TYPE = 'BASE TABLE'
4 ORDER BY TABLE_NAME;
```

Tabelas esperadas:

```
1 | __EFMigrationsHistory
2 |AspNetRoleClaims
3 |AspNetRoles
4 |AspNetUserClaims
5 |AspNetUserLogins
6 |AspNetUserRoles
7 |AspNetUserTokens
8 |AspNetUsers
9 |Categories
10|Games
```

- **RESULTADO ESPERADO:** Todas as tabelas acima devem aparecer na lista. A presença de `__EFMigrationsHistory` confirma que as migrations foram aplicadas. As tabelas `AspNet*` confirmam que o Identity foi configurado corretamente.

■ Passo 4 — Verifique o historico de migrations

SQL

```
1 | SELECT * FROM [__EFMigrationsHistory];
```

Resultado esperado:

1	MigrationId		ProductVersion
2	-----		-----
3	20260521191810_Inicial		8.0.11
4	20260521193849_NomeDaMigration		8.0.11

Ambas as migrations devem estar registradas.

Opcao 2 — Via SQL Server Management Studio (SSMS) ou Azure Data Studio

Se voce tiver o **SQL Server Management Studio** ou o **Azure Data Studio** instalado:

1. Clique em **"Conectar"** (*Connect*) e insira:
 - **Servidor:** `sql-senacgames-luan.database.windows.net,1433`
 - **Autenticacao:** SQL Server Authentication
 - **Login:** `adminsencac`
 - **Senha:** sua senha
2. Expanda o banco `SenacGamesDb` > **Tabelas** para ver todas as tabelas criadas.

- **IMPORTANTE:** O Seed Data (categorias, games e usuario admin) nao sera inserido agora — ele sera executado automaticamente quando a API for iniciada pela primeira vez no Azure. O metodo `SeedData.SeedAsync()` e chamado no `Program.cs` da API durante a inicializacao.

14 Criar o App Service para a SenacGames.API

O que é um Azure App Service?

O **Azure App Service** é o serviço do Azure para hospedar aplicações web. Ele funciona como um servidor web gerenciado — você envia o código da sua aplicação e o Azure cuida de toda a infraestrutura (servidor, sistema operacional, atualizações de segurança, etc.).

Cada App Service possui:

- Uma URL pública no formato `https://nomeapp.azurewebsites.net`
- Um servidor rodando continuamente
- Configurações de ambiente (Application Settings)

Criaremos **dois** App Services — um para a API e outro para a UI.

Passo a passo para criar o App Service da API

■ Passo 1 — Acesse "Serviços de Aplicativos"

1. No Portal Azure, na barra de pesquisa, digite "**Serviços de Aplicativos**" (*App Services*).
2. Clique em "+ Criar" > "Aplicativo Web" (*Web App*).

■ Passo 2 — Aba "Básico" — Configurações gerais

Assinatura (Subscription):

- Selecione: **Azure for Students**

Grupo de recursos (Resource group):

- Selecione: **rg-senacgames**

Nome (Name):

- Digite: `app-senacgames-api-luan` (substitua "luan" pelo seu nome)
- O Azure mostrará a URL: `app-senacgames-api-luan.azurewebsites.net`
- Se o nome estiver disponível, aparecerá uma marca verde.

Publicar (Publish):

- Selecione: "**Código**" (*Code*)

Pilha de runtime (Runtime stack):

- Selecione: **.NET 8 (LTS)**

Sistema operacional (Operating System):

- Selecione: **Windows** ou **Linux**

- DICA: Tanto Windows quanto Linux funcionam para ASP.NET Core 8. O Windows é ligeiramente mais simples para publicar diretamente pelo Visual Studio. Se não tiver preferência, escolha Windows.

Região (Region):

- Selecione a mesma região usada nos recursos anteriores: **Brazil South** ou **East US**

■ Passo 3 — Plano de hospedagem (App Service Plan)

Plano do Windows / Plano do Linux:

- Clique em "**Criar novo**" para criar um plano.
- De um nome ao plano (ex: `plan-senacgames`).

SKU e tamanho:

- Clique em "**Alterar tamanho**" (*Change size*).
- Selecione a opção mais econômica disponível para seu crédito estudantil.

- IMPORTANTE: As opções de plano e preço do Azure mudam frequentemente. Verifique no portal qual é a opção de menor custo disponível no momento. Planos do nível "Gratuito" (Free) ou "Básico" são suficientes para projetos educacionais, quando disponíveis.

■ Passo 4 — Revise e crie

1. Clique em "**Revisar + criar**".
2. Revise todas as configurações.
3. Clique em "**Criar**".
4. Aguarde a implantação (pode levar 1 a 3 minutos).

- RESULTADO ESPERADO: Um App Service chamado `app-senacgames-api-luan` criado e visível no Resource Group `rg-senacgames`. A URL `https://app-senacgames-api-luan.azurewebsites.net` já estará disponível (mas ainda exibirá uma página padrão do Azure até a publicação).

15 Configurar a SenacGames.API no Azure

Antes de publicar a API, precisamos configurar as **Application Settings** do App Service — que é onde ficam as configurações de produção (Connection String, variáveis de ambiente, etc.).

- **CONCEITO IMPORTANTE:** As Application Settings do Azure App Service funcionam como variáveis de ambiente que substituem os valores do `appsettings.json` da aplicação. Isso é seguro porque essas configurações ficam apenas no Azure, não no código-fonte. O ASP.NET Core lê automaticamente essas variáveis de ambiente e as sobrepõem as configurações do `appsettings.json`.

Por que dois underscores (_)?

No `appsettings.json`, as configurações são hierárquicas:

JSON

```
1 {  
2   "ConnectionStrings": {  
3     "DefaultConnection": "valor"  
4   }  
5 }
```

Para representar hierarquia em variáveis de ambiente no formato do Azure, usa-se dois underscores (__) como separador de seção:

```
1 ConnectionStrings__DefaultConnection = valor
```

Passo a passo para configurar as Application Settings

■ Passo 1 — Acesse o App Service da API

1. No Portal Azure, acesse "**Serviços de Aplicativos**".
2. Clique no App Service `app-senacgames-api-luan`.

■ Passo 2 — Acesse Configurações

1. No menu lateral esquerdo, clique em "**Configuração**" (*Configuration*) ou "**Variáveis de ambiente**" (*Environment variables*), dependendo da versão atual do portal.

■ Passo 3 — Adicione a Connection String

Clique em "+ Nova configuracao de aplicativo" (*New application setting*) e adicione:

Nome	Valor
ConnectionStrings__DefaultConnection	Server=tcp:sql-senacgames-luan.database.windows.net,1433;Initial Catalog=SenacGamesDb;Persist Security Info=False;User ID=adminsenac;Password=SuaSenha;MultipleActiveResultSets=False;Encrypt=True;TrustServerCertificate=False;Connection Timeout=30;

- ATENCAO: Substitua adminsenac e SuaSenha pelas credenciais reais que voce criou. Cole a Connection String completa e correta no campo "Valor".

■ Passo 4 — Configure o ambiente de producao

Adicione mais uma configuracao:

Nome	Valor
ASPNETCORE_ENVIRONMENT	Production

- DICA: Definir ASPNETCORE_ENVIRONMENT como Production faz com que o ASP.NET Core: - Nao exiba mensagens de erro detalhadas para o usuario final. - Aplique otimizacoes de performance. - Carregue configuracoes do appsettings.Production.json (se existir).

■ Passo 5 — Salve as configuracoes

1. Clique em "**Salvar**" (*Save*).
2. O Azure perguntara se deseja reiniciar o aplicativo. Confirme clicando em "**Continuar**".

- RESULTADO ESPERADO: A Connection String de producao esta configurada no Azure App Service de forma segura. Quando a API for iniciada, ela usara essa Connection String para se conectar ao Azure SQL Database.

16 Publicar a SenacGames.API pelo Visual Studio

Agora vamos publicar a API no App Service que criamos. Este processo envia o código compilado para o Azure.

Antes de publicar — Verifique o Swagger

Conforme mencionado na seção 10, o Swagger está configurado para rodar apenas em `Development`. Antes de publicar, aplique a Alteração 2 descrita naquela seção no `Program.cs` da API para habilitar o Swagger em produção.

Passo a passo para publicar

■ Passo 1 — Inicie a publicação

1. No **Solution Explorer** do Visual Studio, localize o projeto `SenacGames.API`.
2. Clique com o **botão direito do mouse** sobre o projeto `SenacGames.API`.
3. No menu de contexto que aparecer, clique em **"Publicar..."** (*Publish...*).

■ Passo 2 — Selecione o destino

Uma janela "Publicar" será aberta. Você verá opções de destino:

1. Selecione: **"Azure"**
2. Clique em **"Avançar"** (*Next*).

■ Passo 3 — Selecione o tipo de serviço Azure

1. Selecione: **"Serviço de Aplicativo do Azure (Windows)"** ou **"Serviço de Aplicativo do Azure (Linux)"** — de acordo com o SO que escolheu ao criar o App Service.
2. Clique em **"Avançar"**.

■ Passo 4 — Selecione a assinatura e o recurso

1. No campo **"Assinatura"**, selecione: **Azure for Students**.
2. O Visual Studio listará os App Services disponíveis nessa assinatura.
3. Expanda o grupo de recursos `rg-senacgames`.
4. Selecione o App Service: `app-senacgames-api-luan`.
5. Clique em **"Concluir"** (*Finish*).

■ **DICA:** Se o Visual Studio pedir para fazer login na conta Azure, siga o fluxo de autenticação. Use a mesma conta com a qual ativou o Azure for Students.

■ Passo 5 — Revise o perfil de publicacao

O Visual Studio criara um **Perfil de Publicacao** (*Publish Profile*) com as configuracoes que voce selecionou. Voce vera uma tela de resumo mostrando:

- **Destino:** o App Service selecionado
- **Configuracao:** Release
- **Framework:** net8.0

■ Passo 6 — Publique

1. Clique no botao "**Publicar**" (*Publish*).
2. O Visual Studio ira:
 - **Compilar** o projeto no modo Release.
 - **Empacotar** os arquivos.
 - **Enviar** os arquivos para o Azure App Service.
3. Acompanhe o progresso na janela "**Output**" (*Saida*) na parte inferior do Visual Studio.

■ **DICA:** Se a janela Output nao estiver visivel, acesse: Exibir (View) > Saida (Output).

■ O que esperar ver no Output

```
1 | ----- Build started: Project: SenacGames.API, Configuration: Release -----
2 | ...
3 | Build succeeded.
4 | ...
5 | Publish Succeeded.
6 | Web App was published successfully https://app-senacgames-api-luan.azurewebsites.net
```

■ **RESULTADO ESPERADO:** A mensagem "Publish Succeeded" na janela Output, seguida da URL do App Service. O Visual Studio pode abrir automaticamente a URL no navegador.

■ **ERRO COMUM:** Se aparecer "Build failed", verifique se o projeto SenacGames.API compila sem erros localmente antes de tentar publicar novamente.

17 Testar a API Hospedada

Após a publicação, vamos verificar se a API está funcionando corretamente no Azure.

Acessando a API no Azure

■ Passo 1 — Acesse o Swagger da API

Abra seu navegador e acesse:

```
1 | https://app-senacgames-api-luan.azurewebsites.net/swagger
```

(Substitua `app-senacgames-api-luan` pelo nome real do seu App Service.)

■ RESULTADO ESPERADO: A interface do Swagger abrirá, mostrando todos os endpoints da API: - GET /api/Games - POST /api/Games - GET /api/Games/{id} - PUT /api/Games/{id} - DELETE /api/Games/{id} - GET /api/Categories - POST /api/auth/login - POST /api/auth/logout

■ Passo 2 — Teste o endpoint de listagem de Games

1. Na interface do Swagger, clique em GET /api/Games.
2. Clique em "Try it out" (*Experimental*).
3. Clique em "Execute" (*Executar*).

■ RESULTADO ESPERADO: A API retornará um JSON com a lista de games (inseridos pelo Seed Data durante a primeira inicialização): `json [{ "id": 1, "title": "God of War Ragnarok", ... }, ...]``

■ Verificando se o Seed Data foi executado

O Seed Data é executado automaticamente na primeira inicialização da API (chamado em `Program.cs` pela linha `await SeedData.SeedAsync(app.Services)`). Se os games aparecerem na listagem, o Seed Data funcionou.

Se o endpoint retornar um **array vazio** `[]`, pode significar que o Seed Data ainda não rodou ou encontrou um erro. Reinicie o App Service pelo Portal Azure e verifique os logs.

■ Verificando logs em caso de erro

Se a API retornar um erro HTTP 500 ou a pagina exibir mensagem de erro:

1. No Portal Azure, acesse o App Service `app-senacgames-api-luan`.
2. No menu lateral, clique em "**Fluxo de log**" (*Log stream*) ou "**Diagnostico e solucao de problemas**".
3. Verifique as mensagens de erro para identificar o problema.

Os erros mais comuns neste ponto sao:

- Connection String incorreta nas Application Settings.
 - IP nao autorizado no firewall do Azure SQL.
 - Migrations nao foram aplicadas.
-
-

18 Configurar CORS na API

O que é CORS?

CORS (Cross-Origin Resource Sharing — Compartilhamento de Recursos de Origem Cruzada) é um mecanismo de segurança do navegador que **bloqueia** requisições HTTP feitas de uma URL para outra URL de domínio diferente, a menos que o servidor permita explicitamente.

Exemplo do problema:

```
1 UI: https://app-senacgames-ui-luan.azurewebsites.net
2 API: https://app-senacgames-api-luan.azurewebsites.net
3
4 -> São domínios diferentes!
5 -> O navegador bloqueará as requisições da UI para a API
6 a menos que a API configure CORS corretamente.
```

Como o CORS está configurado atualmente

No `SenacGames.API/Program.cs`, a configuração atual é:

CSHARP

```
1 builder.Services.AddCors(options =>
2 {
3     options.AddPolicy("AllowAll", policy =>
4     {
5         policy.AllowAnyOrigin()
6             .AllowAnyMethod()
7             .AllowAnyHeader();
8     });
9 });
```

E a aplicação da política:

CSHARP

```
1 app.UseCors("AllowAll");
```

Essa configuração permite qualquer origem — o que funciona para o projeto educacional. **Nenhuma** alteração é necessária para este tutorial.

Observação sobre segurança

A politica `AllowAnyOrigin()` é permissiva e aceita requisições de qualquer domínio. Para projetos em produção real (corporativos), recomenda-se restringir as origens específicas. Para este projeto educacional, a configuração atual é adequada e suficiente.

19 Configurar a Comunicação entre UI e API

Esta é uma das etapas mais críticas do deploy. A UI local aponta para a API local (<http://localhost:5223/>). Em produção, ela precisa apontar para a API hospedada no Azure.

Como a UI descobre a URL da API

Conforme analisado no código, a classe `ApiEndpointResolver` tenta resolver a URL da API nesta ordem:

```
1 | 1. Le o launchSettings.json da API (funciona apenas localmente)
2 |   |
3 |   v (Se não encontrar ou não existir...)
4 | 2. Le a seção ApiSettings.BaseUrl do appsettings.json da UI
5 |   |
6 |   v (Se não encontrar...)
7 | 3. Retorna null (a aplicação não conseguiu chamar a API)
```

Em produção no Azure:

- O arquivo `launchSettings.json` **não existe** (não é publicado).
- O resolver tentará ler `ApiSettings.BaseUrl` das variáveis de ambiente (Application Settings do Azure).

Passo 1 — Adicione a seção ApiSettings no appsettings.json da UI

Se ainda não fez a Alteração 1 descrita na seção 10, faça agora.

Abra o arquivo `SenacGames.UI/appsettings.json` e adicione a seção `ApiSettings`:

JSON

```
1 | {
2 |   "ConnectionStrings": {
3 |     "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4 |   },
5 |   "ApiSettings": {
6 |     "BaseUrl": "http://localhost:5223/"
7 |   },
8 |   "Logging": {
9 |     "LogLevel": {
10 |       "Default": "Information",
11 |       "Microsoft.AspNetCore": "Warning"
12 |     }
13 |   },
14 |   "AllowedHosts": "*"
15 | }
```

Salve o arquivo. (Não há credenciais sensíveis aqui — apenas a URL local.)

Passo 2 — Configure a Application Setting na UI (producao)

Apos criar o App Service da UI (proxima secao), precisaremos adicionar a seguinte Application Setting nele:

Nome	Valor
ApiSettings__BaseUrl	https://app-senacgames-api-luan.azurewebsites.net/

- **ATENCAO:** Use dois underscores (__) no nome da configuracao para representar a hierarquia ApiSettings -> BaseUrl. O valor deve ser a URL HTTPS da API hospedada no Azure, com a barra final (/).

Por que NAO usar localhost em producao

- **ERRO MUITO COMUM:** localhost em producao nao e o seu computador. Quando uma aplicacao roda no Azure App Service, localhost se refere ao proprio servidor do Azure onde a aplicacao esta hospedada — nao ao seu computador. Se a UI tentar chamar http://localhost:5223/ no Azure, ela estara tentando chamar a si mesma (ou um endereco inexistente no servidor do Azure), nao a API hospedada. Sempre use a URL publica da API (https://...azurewebsites.net/) nas configuracoes de producao.

Como ficara o fluxo apos a configuracao


```
1 | Azure App Service da UI
2 |   |
3 |   v
4 | ApiEndpointResolver le:
5 | ApiSettings__BaseUrl (via Application Settings do Azure)
6 | = "https://app-senacgames-api-luan.azurewebsites.net/"
7 |   |
8 |   v
9 | HttpClient usa essa URL como BaseAddress
10 |   |
11 |   v
12 | Todas as requisicoes vao para a API hospedada no Azure
13 |   |
14 |   v
15 | API acessa o Azure SQL Database
16 |   |
17 |   v
18 | Dados retornam para a UI
```

20

Criar o App Service para a SenacGames.UI

Agora vamos criar o segundo App Service — para a interface web (MVC).

Por que hospedar separadamente?

A UI e a API são aplicações independentes com responsabilidades diferentes:

Aspecto	SenacGames.UI	SenacGames.API
Tipo	MVC (páginas web)	REST API (dados JSON)
Usuário final acessa	Sim — e a interface	Não diretamente
Acessa o banco	Não — acessa a API	Sim — via EF Core

Hospedar separadamente permite escalar cada parte independentemente e manter responsabilidades claras.

Passo a passo para criar o App Service da UI

■ Passo 1 — Acesse "Serviços de Aplicativos"

1. No Portal Azure, busque por "Serviços de Aplicativos".
2. Clique em "+ Criar" > "Aplicativo Web".

■ Passo 2 — Preencha as configurações

Assinatura: Azure for Students

Grupo de recursos: rg-senacgames

Nome: app-senacgames-ui-luan (substitua "luan" pelo seu nome)

A URL gerada será: `https://app-senacgames-ui-luan.azurewebsites.net`

Publicar: Código (Code)

Pilha de runtime: .NET 8 (LTS)

Sistema operacional: Windows (ou Linux — o mesmo escolhido para a API)

Região: Mesma região dos outros recursos

Plano: Pode usar o **mesmo plano** criado para a API (`plan-senacgames`) para economizar — basta selecionar o plano existente em vez de criar um novo.

■ Passo 3 — Crie o App Service

1. Clique em **"Revisar + criar"**.
2. Revise as informacoes.
3. Clique em **"Criar"**.

■ **RESULTADO ESPERADO:** Um segundo App Service app-senacgames-ui-luan criado no Resource Group rg-senacgames.

21 Configurar a SenacGames.UI no Azure

Antes de publicar a UI, configure as Application Settings do App Service da UI.

Passo a passo para configurar

■ Passo 1 — Acesse o App Service da UI

1. No Portal Azure, acesse **"Serviços de Aplicativos"**.
2. Clique em `app-senacgames-ui-luan`.

■ Passo 2 — Acesse Configuracoes/Variaveis de ambiente

1. No menu lateral esquerdo, clique em **"Configuracao"** (*Configuration*) ou **"Variaveis de ambiente"** (*Environment variables*).

■ Passo 3 — Adicione as configuracoes necessarias

Adicione as seguintes configuracoes clicando em **" + Nova configuracao de aplicativo "** para cada uma:

Nome	Valor
<code>ApiSettings__BaseUrl</code>	<code>https://app-senacgames-api-luan.azurewebsites.net/</code>
<code>ASPNETCORE_ENVIRONMENT</code>	<code>Production</code>

- **ATENCAO CRITICA:** - Substitua `app-senacgames-api-luan` pelo nome real do seu App Service da API. - Inclua a barra final na URL: `https://...azurewebsites.net/` - Use HTTPS (nao HTTP) para comunicacao segura entre os servicos.

■ Passo 4 — Salve as configuracoes

1. Clique em **"Salvar"**.
2. Confirme o reinicio clicando em **"Continuar"**.

- **RESULTADO ESPERADO:** A UI esta configurada para apontar para a URL HTTPS da API hospedada no Azure. Quando a UI for iniciada no Azure, o `ApiEndpointResolver` lerao `ApiSettings__BaseUrl` das variaveis de ambiente (Application Settings) e usara essa URL para todas as chamadas HTTP para a API.

22 Publicar a SenacGames.UI pelo Visual Studio

Antes de publicar — Verifique o appsettings.json

Certifique-se de que o `SenacGames.UI/appsettings.json` contem a seção `ApiSettings` (adicionada na seção 19):

JSON

```
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=SenacGamesDb;Trusted_C
4    },
5    "ApiSettings": {
6      "BaseUrl": "http://localhost:5223/"
7    },
8    "Logging": {
9      "LogLevel": {
10       "Default": "Information",
11       "Microsoft.AspNetCore": "Warning"
12     }
13   },
14   "AllowedHosts": "*"
15 }
```

Passo a passo para publicar a UI

■ Passo 1 — Inicie a publicação

1. No **Solution Explorer**, clique com o **botão direito do mouse** sobre o projeto `SenacGames.UI`.
2. Clique em **"Publicar..."** (*Publish...*).

■ Passo 2 — Selecione o destino

1. Selecione: **"Azure"**
2. Clique em **"Avançar"**.

■ Passo 3 — Selecione o tipo de serviço

1. Selecione: **"Serviço de Aplicativo do Azure (Windows)"** (ou Linux, conforme seu App Service).
2. Clique em **"Avançar"**.

■ Passo 4 — Selecione o App Service da UI

1. **Assinatura:** Azure for Students.

2. Expanda o grupo de recursos `rg-senacgames`.
3. Selecione: `app-senacgames-ui-luan`.
4. Clique em **"Concluir"**.

■ Passo 5 — Publique

1. Clique em **"Publicar"**.
2. Aguarde a compilacao e o envio dos arquivos.
3. Acompanhe o progresso na janela **Output**.

■ RESULTADO ESPERADO: ` Publish Succeeded. Web App was published successfully
https://app-senacgames-ui-luan.azurewebsites.net `

■ Passo 6 — Acesse a UI no navegador

O Visual Studio pode abrir automaticamente a URL no navegador. Caso contrario, acesse manualmente:

```
1 | https://app-senacgames-ui-luan.azurewebsites.net
```

■ RESULTADO ESPERADO: A pagina inicial do SenacGames abrira no navegador, com a listagem de games carregada da API hospedada no Azure.

23 Teste Completo da Aplicação em Produção

Após publicar ambas as aplicações, realize um teste completo para garantir que tudo funciona corretamente em produção.

Checklist de validação

```
1  INFRAESTRUTURA
2  [ ] Resource Group rg-senacgames contem todos os recursos
3  [ ] Azure SQL Database SenacGamesDb existe e tem tabelas
4  [ ] App Service da API esta em execucao
5  [ ] App Service da UI esta em execucao
6
7  API
8  [ ] URL https://app-senacgames-api-luan.azurewebsites.net/swagger abre o Swagger
9  [ ] GET /api/Games retorna a lista de games (com dados do Seed)
10 [ ] GET /api/Categories retorna as categorias
11 [ ] POST /api/auth/login funciona com admin@senacgames.com / Admin@123
12 [ ] A API retorna HTTPS (nao HTTP)
13
14 UI
15 [ ] URL https://app-senacgames-ui-luan.azurewebsites.net abre a pagina inicial
16 [ ] A pagina inicial exibe a lista de games
17 [ ] As imagens dos games carregam corretamente
18 [ ] As categorias aparecem corretamente
19
20 AUTENTICACAO
21 [ ] A pagina de login abre
22 [ ] Login com admin@senacgames.com / Admin@123 funciona
23 [ ] Após login, o menu de admin aparece
24 [ ] Logout funciona
25
26 CRUD
27 [ ] Criar um novo game funciona
28 [ ] Editar um game existente funciona
29 [ ] Excluir um game funciona
30 [ ] As alteracoes persistem (ficam salvas no Azure SQL)
31
32 SEGURANCA
33 [ ] Todas as URLs usam HTTPS
34 [ ] A UI nao contem referencias a localhost em producao
35 [ ] Usuarios nao autenticados nao conseguem acessar rotas protegidas
```


24 Fluxo Completo de Comunicação

Diagrama do fluxo em producao

```

1      USUARIO NO NAVEGADOR
2      |
3      | HTTPS
4      v
5      +-----+
6      | SenacGames.UI |
7      | Azure App Service |
8      | ASP.NET Core MVC |
9      | |
10     | Controller MVC |
11     | -> Chama HttpGameService |
12     +-----+
13     |
14     | HTTPS (HttpClient)
15     | ApiSettings__BaseUrl
16     v
17     +-----+
18     | SenacGames.API |
19     | Azure App Service |
20     | ASP.NET Core Web API |
21     | |
22     | GameController |
23     | -> IGameService |
24     | -> IGameRepository |
25     +-----+
26     |
27     | Entity Framework Core
28     | Connection String -> Azure SQL
29     v
30     +-----+
31     | Azure SQL Database |
32     | SenacGamesDb |
33     | Tabelas: Games, |
34     | Categories, AspNet* |
35     +-----+

```

Exemplo completo de uma requisicao

Veja o caminho completo quando um usuario acessa a listagem de games:

```
1 1. Usuario acessa:
2   https://app-senacgames-ui-luan.azurewebsites.net/Games
3
4 2. Azure App Service da UI recebe a requisicao HTTP
5
6 3. GameController (UI) e invocado, metodo Index()
7
8 4. HttpGameService e chamado
9   -> HttpClient com BaseAddress = API hospedada
10  -> Envia: GET https://app-senacgames-api-luan.azurewebsites.net/api/games
11
12 5. Azure App Service da API recebe a requisicao
13
14 6. GameController (API) e invocado, metodo GetAll()
15
16 7. IGameService e chamado -> GameService
17
18 8. IGameRepository e chamado -> GameRepository
19
20 9. Entity Framework Core executa:
21   SELECT * FROM Games
22   (no Azure SQL Database SenacGamesDb)
23
24 10. Azure SQL retorna os dados
25
26 11. Repository retorna List<Game>
27
28 12. Service retorna List<GameViewModel>
29
30 13. API serializa como JSON e responde com HTTP 200
31
32 14. HttpGameService na UI recebe o JSON
33
34 15. GameController da UI passa os dados para a View
35
36 16. Razor renderiza o HTML com a lista de games
37
38 17. HTML e enviado ao navegador do usuario
39
40 18. Usuario ve a pagina com os games listados
```

25 Problemas Comuns e Soluções

Connection String nao funciona

Sintomas: Erro ao executar `Update-Database` ou erros ao iniciar a API no Azure.

Causas possiveis e solucoes:

Causa	Solucao
Usuario incorreto	Verifique o login do administrador SQL criado
Senha incorreta	Confirme a senha (respeitando maiusculas/minusculas)
Nome do servidor incorreto	Copie o nome exato do Portal Azure
Banco incorreto	Confirme que o nome e <code>SenacGamesDb</code>
Firewall bloqueando	Adicione seu IP atual nas regras de firewall do Azure SQL
Application Setting nao salva	Confirme que clicou em "Salvar" apos adicionar as configuracoes
Formato errado de configuracao	Use <code>__</code> duplo, nao <code>.</code> para hierarquia nas Application Settings

Login failed for user 'adminsenc'

Sintomas: Erro `Login failed for user 'adminsenc'` ao executar migrations ou ao iniciar a API.

Causas e solucoes:

- 1. Senha incorreta:** Verifique se a senha esta correta. Senhas SQL sao case-sensitive.
- 2. Usuario incorreto:** Confira o nome exato do usuario no Portal Azure.
- 3. Connection String incompleta:** Certifique-se de que a Connection String esta completa e sem caracteres cortados.

Cannot open server

Sintomas: Erro `Cannot open server 'sql-senacgames-luan' requested by the login` ao tentar conectar.

Causa: O firewall do Azure SQL esta bloqueando o seu IP atual.

Solucao:

1. Acesse o Portal Azure > SQL Server > Rede/Firewall.
2. Clique em "Adicionar IP do cliente".
3. Salve e tente novamente.

API funciona localmente mas nao no Azure

Investigue nesta ordem:

1. **Verifique os logs** do App Service no Portal Azure > Log Stream.
2. **Verifique as Application Settings:** a Connection String esta correta?
3. **Verifique o ambiente:** `ASPNETCORE_ENVIRONMENT` esta configurado?
4. **Verifique o Runtime:** o App Service usa .NET 8?
5. **Verifique o firewall:** a opcao "Permitir servicos Azure" esta ativada no SQL Server?

UI funciona localmente mas nao no Azure

Investigue:

1. `ApiSettings__BaseUrl` **esta configurado** nas Application Settings da UI?
2. A URL aponta para **HTTPS** e nao para HTTP?
3. A URL e a URL real da API hospedada, nao `localhost`?
4. A barra final esta presente? Ex: `https://...azurewebsites.net/`
5. **Verifique os logs** da UI no Portal Azure.

UI nao consegue acessar a API

Investigue:

1. **CORS:** a politica `AllowAll` esta aplicada na API? (`app.UseCors("AllowAll")`).
2. **URL:** a URL da API nas configuracoes da UI esta correta?
3. **HTTPS:** ambos os servicos usam HTTPS?
4. **API esta em execucao:** acesse a URL da API diretamente no navegador para confirmar.
5. **App Service em execucao:** no Portal Azure, confirme que o App Service da API esta com status "Em execucao" (*Running*).

Banco nao possui tabelas

Causa: As migrations nao foram aplicadas ao Azure SQL.

Solucao:

1. Confirme que a Connection String no `appsettings.json` da API aponta para o Azure SQL.

2. Execute novamente:

POWERSHELL

```
1 | Update-Database -Project SenacGames.Infrastructure -StartupProject SenacGames.API
```

3. Verifique os erros no Console do Gerenciador de Pacotes.

Update-Database criou banco local novamente (LocalDB)

Causa: A Connection String ainda aponta para (localdb)\mssqllocaldb.

Solucao:

1. Abra `SenacGames.API/appsettings.json`.
2. Confirme que a Connection String foi substituida pela do Azure SQL.
3. Execute o `Update-Database` novamente.
4. Apos o sucesso, reverta para a Connection String do LocalDB.

Publicacao do Visual Studio falhou

Possiveis causas e solucoes:

Causa	Solucao
Nao esta logado na conta Azure	Acesse Ferramentas > Opcoes > Azure Service Authentication
Sessao expirada	Faca logout e login novamente no Visual Studio
Projeto nao compila	Corrija todos os erros de build antes de publicar
App Service nao encontrado	Confirme a assinatura e o nome do App Service
Perfil de publicacao corrompido	Delete o arquivo <code>.pubxml</code> e crie um novo

Como deletar o perfil e recriar:

1. No Solution Explorer, expanda `SenacGames.API > Properties > PublishProfiles`.
2. Delete o arquivo `*.pubxml` existente.
3. Repita o processo de publicacao a partir do Passo 1 (secao 16).

Seed Data nao foi executado (Games nao aparecem)

Causa: O `SeedData.SeedAsync()` e chamado no `Program.cs` da API na primeira inicializacao. Se a API encontrou um erro ao iniciar (ex: Connection String errada), o Seed nao foi executado.

Solucao:

1. Corrija o erro que impediu a API de inicializar corretamente.
2. Reinicie o App Service da API no Portal Azure (menu lateral > **"Visao Geral"** > botao **"Reiniciar"**).
3. Verifique os logs para confirmar que a inicializacao foi bem-sucedida.

Erro HTTP 500 na UI apos login

Causa: Provavelmente a UI nao consegue se comunicar com a API apos o login.

Investigue:

1. A `ApiSettings__BaseUrl` esta configurada corretamente na UI?
2. A API esta acessivel pelo Swagger?
3. Verifique os logs da UI.

26 Monitoramento e Logs

Por que monitorar?

```
1  "Funciona no meu computador"
2
3  nao e igual a
4
5  "Funcionara no Azure"
```

O ambiente do Azure é diferente do seu computador local. Pode haver diferenças de configuração, versão, permissões, etc. Os logs são sua principal ferramenta de diagnóstico.

Acessando os logs do App Service

■ Log Stream (em tempo real)

O Log Stream permite ver os logs da aplicação em tempo real.

1. No Portal Azure, acesse o App Service (da API ou da UI).
2. No menu lateral esquerdo, procure por **"Fluxo de log"** (*Log stream*).
3. Clique para abrir.
4. Os logs aparecerão em tempo real enquanto a aplicação recebe requisições.

■ **IMPORTANTE:** Para o Log Stream funcionar, é necessário ativar o logging do App Service: 1. No menu lateral, clique em "Logs do Serviço de Aplicativo" (App Service Logs). 2. Ative "Registro em log do aplicativo (Sistema de Arquivos)" (Application logging - File System). 3. Defina o nível para "Informações" (Information). 4. Clique em "Salvar".

■ Diagnóstico e solução de problemas

1. No menu lateral do App Service, clique em **"Diagnosticar e solucionar problemas"** (*Diagnose and solve problems*).
2. Selecione categorias como **"Falhas na Disponibilidade e no Desempenho"** para diagnósticos automáticos.

Interpretando erros comuns nos logs

Mensagem no log	O que significa
A network-related error occurred while establishing a connection	Problema de Connection String ou Firewall
The connection property has not been initialized	Connection String ausente ou malformatada
No such host is known	Nome do servidor SQL incorreto
Login failed for user	Credenciais incorretas
Cannot open database "SenacGamesDb"	Banco de dados nao existe ou Connection String errada
An error occurred while applying the migration	Problema ao executar migrations automaticas do SeedData

27 Segurança e Boas Práticas

Nunca exponha credenciais em repositórios públicos

- **ATENCAO CRITICA:** Uma das violacoes de seguranca mais comuns cometidas por desenvolvedores iniciantes e fazer commit de senhas e Connection Strings com credenciais reais para o GitHub. Consequencias: - Qualquer pessoa no mundo pode ver e usar suas credenciais. - Robos automatizados varrem o GitHub continuamente em busca de credenciais expostas. - Sua conta Azure pode ser comprometida e usada para fins maliciosos (gerando custos enormes).

Regras de ouro:

1. **Connection Strings de producao** --> somente nas Application Settings do Azure.
2. **Senhas** --> nunca no codigo, sempre em variaveis de ambiente.
3. **appsettings.json** --> pode estar no GitHub somente com valores locais/padrao sem credenciais.
4. **.gitignore** --> confirme que **appsettings.Production.json** esta listado se voce o criar.

Configuracoes de seguranca recomendadas

Pratica	Status no projeto	Observacao
Senhas nao estao no codigo	Configuradas no Azure App Service	Correto
HTTPS habilitado	App Service ativa HTTPS automaticamente	Correto
Senhas de usuario com requisitos	Configurado no Identity (API Program.cs)	Correto
Firewall do SQL configurado	Apos as configuracoes deste manual	Verifique
CORS configurado	Politica AllowAll (para fins educacionais)	Adequado para este projeto

Boas praticas gerais

- Utilize **HTTPS** sempre que possivel (o Azure App Service ativa HTTPS por padrao).
- Nao desabilite a validacao de certificados em producao (nunca use `TrustServerCertificate=True` em producao sem motivo especifico).
- **Remova regras de firewall desnecessarias** apos o projeto (ex: se adicionou um IP temporario para migration, remova depois).

- **Monitore o consumo de créditos** regularmente.
 - Não use a mesma senha para banco de dados, conta Azure e outros serviços.
 - **Documente as credenciais** em local seguro e privado (gerenciador de senhas, não em arquivos de texto simples).
-
-

28 Encerramento do Projeto e Economia de Créditos

Ao final do projeto (ou semestre), é importante encerrar os recursos para não continuar consumindo créditos.

Como identificar recursos ativos

1. Acesse <https://portal.azure.com>.
2. Na barra de pesquisa, acesse "**Grupos de recursos**".
3. Clique no grupo `rg-senacgames`.
4. Todos os recursos do projeto serão listados com seu status.

Como excluir todos os recursos de uma vez

A forma mais eficiente de encerrar o projeto é **excluir o Resource Group completo**. Isso remove automaticamente todos os recursos dentro dele.

■ **ATENÇÃO CRÍTICA:** A exclusão do Resource Group é irreversível. Todos os dados do banco de dados, configurações e arquivos publicados serão permanentemente deletados.

Checklist antes de excluir

- | | | | |
|---|--|--------------------------|--|
| 1 | | ☐ | Fiz backup de quaisquer dados importantes do banco (se necessário) |
| 2 | | ☐ | Confirmei que não precisarei mais dos recursos do Azure |
| 3 | | ☐ | Anotei as URLs e configurações para referência futura |
| 4 | | ☐ | Verifiquei que o Resource Group correto será excluído |

Passo a passo para excluir o Resource Group

1. No Portal Azure, acesse "**Grupos de recursos**".
2. Clique em `rg-senacgames`.
3. No topo da página, clique em "**Excluir grupo de recursos**" (*Delete resource group*).
4. O Azure pedirá que você **digite o nome do Resource Group** como confirmação.
5. Digite: `rg-senacgames`
6. Clique em "**Excluir**" (*Delete*).
7. Aguarde a conclusão (pode levar alguns minutos).

- **RESULTADO ESPERADO:** Todos os recursos do SenacGames (banco de dados, app services, servidor SQL) são removidos e o consumo de créditos cessa.

Como verificar que não há mais recursos ativos

Após a exclusão:

1. Acesse **Cost Management** no Portal Azure.
2. Verifique a **"Análise de custos"** dos próximos dias.
3. O consumo deve cair a zero para os recursos excluídos.
4. Verifique também se não há outros recursos em outras regiões ou grupos.

29 Checklist Final

Use este checklist como referencia rapida para verificar se completou todas as etapas.

AZURE — Infraestrutura

- 1 [] Conta Microsoft criada e acessivel
- 2 [] Azure for Students ativado com e-mail @senacedu.sp.br
- 3 [] Creditos confirmados no portal Azure
- 4 [] Resource Group rg-senacgames criado
- 5 [] SQL Server logico criado (ex: sql-senacgames-luan.database.windows.net)
- 6 [] SQL Database SenacGamesDb criado dentro do SQL Server
- 7 [] Firewall: IP do computador adicionado
- 8 [] Firewall: Acesso de servicos Azure ativado

BANCO DE DADOS

- 1 [] Connection String do Azure SQL copiada e guardada com seguranca
- 2 [] appsettings.json da API configurado temporariamente com Connection String Azure
- 3 [] Update-Database executado com sucesso
- 4 [] Tabelas verificadas no Portal Azure (Query Editor)
- 5 [] __EFMigrationsHistory contem as duas migrations
- 6 [] Tabelas ASPNet* criadas corretamente
- 7 [] Tabelas Games e Categories criadas
- 8 [] appsettings.json da API revertido para LocalDB apos as migrations

SenacGames.API — Publicacao

- 1 [] appsettings.json da UI contem ApiSettings:BaseUrl (secao 19)
- 2 [] Swagger habilitado em producao no Program.cs (secao 10, Alteracao 2)
- 3 [] App Service app-senacgames-api-luan criado
- 4 [] Application Setting: ConnectionStrings__DefaultConnection configurada no Azure
- 5 [] Application Setting: ASPNETCORE_ENVIRONMENT = Production configurada
- 6 [] API publicada pelo Visual Studio com sucesso
- 7 [] https://...azurewebsites.net/swagger abre corretamente
- 8 [] GET /api/Games retorna dados (Seed Data funcionou)
- 9 [] GET /api/Categories retorna dados

SenacGames.UI — Publicacao

```
1 [ ] appsettings.json da UI contem ApiSettings:BaseUrl com URL local
2 [ ] App Service app-senacgames-ui-luan criado
3 [ ] Application Setting: ApiSettings__BaseUrl = URL HTTPS da API configurada no Azure
4 [ ] Application Setting: ASPNETCORE_ENVIRONMENT = Production configurada
5 [ ] UI publicada pelo Visual Studio com sucesso
6 [ ] https://...azurewebsites.net abre a pagina inicial
7 [ ] Listagem de games carrega corretamente
```

VALIDACAO COMPLETA

```
1 [ ] UI consegue buscar dados da API hospedada
2 [ ] API conecta e consulta o Azure SQL Database
3 [ ] Login funciona (admin@senacgames.com / Admin@123)
4 [ ] CRUD completo funciona (criar, editar, excluir games)
5 [ ] Dados persistem entre sessoes (ficam salvos no Azure SQL)
6 [ ] HTTPS funciona em ambas as aplicacoes
7 [ ] Nenhuma referencia incorreta a localhost em producao
8 [ ] Credenciais nao estao expostas em codigo ou no GitHub
```

SEGURANCA

```
1 [ ] Connection String de producao APENAS no Azure App Service
2 [ ] .gitignore nao inclui arquivos com credenciais de producao
3 [ ] Regras de firewall desnecessarias removidas
4 [ ] Alertas de custo configurados no Azure
```

Referência Rápida de Comandos

Entity Framework Core — Package Manager Console (Visual Studio)

POWERSHELL

```
1 # Aplicar migrations ao banco (Azure SQL)
2 Update-Database -Project SenacGames.Infrastructure -StartupProject SenacGames.API
3
4 # Criar nova migration (quando necessário no futuro)
5 Add-Migration NomeDaMigration -Project SenacGames.Infrastructure -StartupProject SenacGa
6
7 # Listar migrations existentes
8 Get-Migration -Project SenacGames.Infrastructure -StartupProject SenacGames.API
```

Entity Framework Core — Terminal (dotnet CLI)

POWERSHELL

```
1 # Aplicar migrations ao banco (Azure SQL)
2 dotnet ef database update --project SenacGames.Infrastructure --startup-project SenacGam
3
4 # Instalar ferramentas EF Core globalmente (se necessário)
5 dotnet tool install --global dotnet-ef
```

Informações de Referência do Projeto

.NET Version	.NET 8 (net8.0)
DbContext	SenacGamesDbContext em SenacGames.Infrastructure/Context/
Migrations	SenacGames.Infrastructure/Migrations/
Startup Project (EF)	SenacGames.API
Project (EF)	SenacGames.Infrastructure
Connection String (local)	Server=(localdb)\mssqllocaldb;Database=SenacGamesDb;...
Connection String Key	ConnectionStrings:DefaultConnection
URL da API (local)	http://localhost:5223/
URL da API (Azure)	https://app-senacgames-api-{nome}.azurewebsites.net/
URL da UI (Azure)	https://app-senacgames-ui-{nome}.azurewebsites.net/
Seed Data executado em	SenacGames.API/Program.cs via SeedData.SeedAsync()
Usuário admin padrão	admin@senacgames.com / Admin@123

Variável de ambiente da URL da API	ApiSettings__BaseUrl
Variável da Connection String	ConnectionStrings__DefaultConnection

Manual criado para uso educacional na disciplina de Desenvolvimento Web com ASP.NET Core. Baseado na análise real da solução SenacGames — .NET 8 — Entity Framework Core 8.0.11.